

Part 1: The Phantom Bug

Total points: 50000000
Total hits: 34471028
Estimated PI: 2.75768224
Time: 4494 ms

Total points: 50000000
Total hits: 31238219
Estimated PI: 2.49905752
Time: 2465 ms

Total points: 50000000
Total hits: 31303390
Estimated PI: 2.5042712
Time: 2572 ms

Total points: 50000000
Total hits: 33545176
Estimated PI: 2.68361408
Time: 3192 ms

Total points: 50000000
Total hits: 36363412
Estimated PI: 2.90907296
Time: 3834 ms

Part 2: The Synchronization Trap

=== Single-threaded ===
Pi = 3.14158368
Time = 0.981891898 seconds

=== 4 Threads + synchronized ===
Pi = 3.14164408
Time = 14.454483936 seconds

=== 4 Threads + AtomicLong ===
Pi = 3.1415492
Time = 18.863446491 seconds

Part 3: OpenMP-Style Reduction

OpenMP-Style Reduction Benchmark
Iterations: 100000000

Threads: 1 | Pi: 3.141549 | Time: 3.2520 seconds
Threads: 2 | Pi: 3.141583 | Time: 13.0531 seconds

Threads: 4 | Pi: 3.141549 | Time: 19.4414 seconds
Threads: 8 | Pi: 3.141534 | Time: 32.7994 seconds
Threads: 16 | Pi: 3.141710 | Time: 21.8911 seconds
Threads: 32 | Pi: 3.141995 | Time: 25.7090 seconds

Threads (T)	Runtime (ms)	Speedup vs. 1 Thread	Efficiency (T×Speedup)
1	3252.0	1.000x	100.0%
2	13053.1	0.249x	12.5%
4	19441.4	0.167x	4.2%
8	32799.4	0.099x	1.2%
16	21891.1	0.149x	0.9%
32	25709.0	0.126x	0.4%

1. Why didn't the 8-core CPU run twice as fast with 16 threads compared to 8 threads?

Because the CPU has only 8 physical cores, so when using 16 threads, multiple threads have to share the same physical cores. The operating system schedules the extra threads on the available cores instead of giving every thread its own core.

Also, thread management and context switching create overhead. Therefore, increasing the number of threads from 8 to 16 does not automatically double performance. In this case, 8 threads can already keep the 8 cores busy, so additional threads mainly add overhead.

2. Why was the synchronized version in Part 2 slower than running on one single core?

The synchronized version was slower because all threads had to compete for access to the shared variable.

Every time a thread updates the shared counter, it must acquire a lock. While one thread holds the lock, the other threads have to wait. This creates:

- Lock acquisition/release overhead
- Threads waiting for the lock
- Context switching
- Contention between threads

As a result, the program spends a lot of time synchronizing instead of doing useful calculations. With only one thread, there is no competition for the lock, so it can run more efficiently.

